Studio Nx

P1-04 회원가입 API — 함수 설명 합본 (14~25)

2026-10-07

목차

1. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/14_security_check_password_rules.md
2. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/15_security_hash_password.md
3. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/16_security_verify_password.md
4. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/17_auth_service_normalize_email.md
5. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/18_auth_service_release_withdrawn.md
6. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/19_auth_service_check_email.md
7. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/20_auth_service_check_nickname.md
8. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/21_auth_service_signup.md
9. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/22_auth_route_signup.md
10. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/23_auth_route_check_email.md
11. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/24_auth_route_check_nickname.md
12. ai-notes/backend(fastAPI_Python)/P1-04 - 회원가입 API/25_auth_schema_signup_request.md

Backend(FastAPI/Python) 14 — `check_password_rules()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입 — 비밀번호 규칙 · PRD 01 FR-102
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	13 모델 관계 (P1-03 끝) · 15 `hash_password()`

1. 하는 일

비밀번호가 규칙 6가지 중 **무엇을 어겼는지** 이름 목록으로 돌려준다. 빈 목록이면 통과다.

`check_password_rules("abcdefgh")` → `["upper", "number", "special"]`

이름	규칙	join.html 조건 목록의 줄
`length`	8 ~ 20자	8–20 characters
`upper`	대문자 1개 이상	Uppercase letter
`lower`	소문자 1개 이상	Lowercase letter
`number`	숫자 1개 이상	Number
`special`	키보드 기호 1개 이상 (`!@#$%^&*` 등)	Special character (!@#$…)
`allowed`	영문 · 숫자 · 키보드 기호만 (한글 · 공백 불가)	(목록에 없음 — 칸 아래 오류 문구)

2. 언제 쓰나

쓰는 곳	하는 일
P1-04 `signup()` (08)	가입 요청의 비밀번호 검사. 실패하면 422
P1-09 `change_password()`	새 비밀번호 검사 (같은 규칙)

브라우저도 같은 규칙으로 검사한다 (`join.js` 의 `PASSWORD_RULES`, P1-07). 입력할 때마다 조건 목록에 ✓ 를 붙인다. **서버가 한 번 더 검사하는 이유:** 브라우저 검사는 개발자 도구나 직접 만든 요청으로 건너뛸 수 있다.

실패하면 `signup()` 이 이 목록을 그대로 담아 응답한다.

```
422 {"error": {"code": "VALIDATION_ERROR", "message": "Check the password requirements.",
               "details": {"password": ["upper", "number", "special"]}}}
```

화면은 `details.password` 의 이름으로 join.html 조건 목록의 해당 줄(`li[data-rule="upper"]` 등)을 표시할 수 있다.

3. 코드

import (파일 맨 위)

```
import re
import string

import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다. **`re` · `string` 두 줄을 이번에 추가했고**, 이 함수는 그 둘만 쓴다. 둘 다 파이썬 기본 모듈이라 설치가 필요 없다.

import	가져오는 것	쓰는 곳
`re`	정규식 (글자 패턴 찾기)	대문자 · 소문자 · 숫자 · 허용 문자 검사
`string`	자주 쓰는 글자 모음	`string.punctuation` — 키보드 기호 32개

함수

```
PASSWORD_MIN, PASSWORD_MAX = 8, 20


def check_password_rules(plain: str) -> list[str]:
    failed: list[str] = []
    if not PASSWORD_MIN <= len(plain) <= PASSWORD_MAX:
        failed.append("length")
    if not re.search(r"[A-Z]", plain):
        failed.append("upper")
    if not re.search(r"[a-z]", plain):
        failed.append("lower")
    if not re.search(r"[0-9]", plain):
        failed.append("number")
    if not any(ch in string.punctuation for ch in plain):
        failed.append("special")
    if not re.fullmatch(r"[!-~]+", plain):
        failed.append("allowed")
    return failed
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와 `모듈.이름` 으로 씀 (예: `re.search`, `string.punctuation`)

식	뜻
`PASSWORD_MIN, PASSWORD_MAX = 8, 20`	두 변수에 한 번에 넣음 (`PASSWORD_MIN = 8`, `PASSWORD_MAX = 20`)
`8 <= len(plain) <= 20`	파이썬은 비교를 이어 쓸 수 있음. `8 <= len(plain) and len(plain) <= 20` 과 같음
`r"[A-Z]"`	정규식. `[A-Z]` = A 부터 Z 중 한 글자. 앞의 `r` 은 `\` 를 글자 그대로 쓰겠다는 표시
`re.search(패턴, 글)`	글 **어딘가에** 패턴이 있으면 결과, 없으면 `None`. `not None` 은 `True`
`re.fullmatch(패턴, 글)`	글 **전체가** 패턴과 맞아야 결과

식	뜻
`[!-~]+`	`!`(33번) 부터 `~`(126번) 까지의 글자가 1개 이상. 영문 · 숫자 · 키보드 기호가 모두 이 범위다. 한글 · 공백(32번)은 범위 밖
`any(ch in string.punctuation for ch in plain)`	글자를 하나씩 보며 기호인 것이 **하나라도** 있으면 `True`

입력	결과
`"Abcdef1!"`	`[]` (통과)
`"Ab1!xyz"` (7자)	`["length"]`
`"abcdefgh"`	`["upper", "number", "special"]`
`"Abcdef1!한"`	`["allowed"]`
`"Abc def1!"` (공백)	`["allowed"]`
`""`	6개 모두

4. 흐름

1. 빈 목록 `failed` 를 만든다.
2. 규칙 6개를 **차례대로 모두** 검사하고, 어긴 것의 이름을 넣는다.
3. 목록을 돌려준다.

5. 왜 이렇게

선택	이유
처음 실패에서 멈추지 않고 모두 검사	화면이 어긴 조건을 한 번에 모두 표시할 수 있다
`True/False` 가 아니라 이름 목록	화면의 조건 줄 이름(`data-rule`)과 같아서 그대로 연결된다
이 함수는 오류를 던지지 않음	가입(08)과 비밀번호 변경(P1-09)이 같은 함수를 쓰고, 오류 응답은 각자 만든다
기호는 `string.punctuation`	파이썬이 정해 둔 키보드 기호 32개(`!"#$%&'()*+,-./:;<=>?@[\]^_{|}~`)다. `join.js` 의 `/[!-/:-@[-{-~]/` 와 같은 모음인지 시험으로 확인했다
최대 20자	PRD FR-102. bcrypt(15)는 72바이트까지만 쓰므로 그보다 충분히 짧다

6. 시험 (`tests/test_password.py`, 새로 만듦 · 16개)

시험	확인한 것
통과 4개	8자 · 20자 · 반복 글자 · 밑줄 기호 → `[]`
실패 10개	7자 · 21자 · 대문자 없음 · 소문자 없음 · 숫자 없음 · 기호 없음 · 한글 · 공백 · 여러 개 동시 · 빈 글자 → 위 표의 목록
모든 기호	`string.punctuation` 32개 각각이 `special` 로 인정됨
화면과 같은 규칙	ASCII 33 ~ 126 에서 영문 · 숫자를 뺀 모음 = `string.punctuation` (= `join.js` 정규식)

전체 시험 92개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 15 — `hash_password()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입 — 비밀번호 해시 저장 · PRD 01 FR-102
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	14 check_password_rules() · 16 verify_password()

1. 하는 일

비밀번호를 **되돌릴 수 없는 글자(해시)** 로 바꾼다. DB 에는 비밀번호 대신 이 해시만 저장한다.

`hash_password("Abcdef1!")` → `"$2b$12$Q9xK…(60자)"`

해시의 부분	예	뜻
`$2b$`		bcrypt 방식
`12$`		비용(cost) 12 — 계산을 2^{12} 번 반복. 클수록 느림
다음 22자	`Q9xKc3…`	소금(salt) — 해시마다 새로 만든 무작위 값
나머지 31자		비밀번호 + 소금으로 계산한 결과

2. 언제 쓰나

쓰는 곳	하는 일
P1-04 `signup()` (21)	규칙 검사(14)를 통과한 비밀번호를 해시해 `users.password_hash` 에 저장
P1-09 `change_password()`	새 비밀번호를 해시해 바꿔 저장

로그인(P1-05)할 때는 이 해시를 되돌리지 않는다. 입력한 비밀번호를 같은 소금으로 다시 계산해 결과가 같은지 비교한다 (다음 16 `verify_password()`).

왜 비밀번호를 그대로 저장하지 않나

저장 방식	DB 가 유출되면
비밀번호 그대로	모든 회원의 비밀번호가 바로 드러남. 같은 비밀번호를 쓰는 다른 사이트까지 위험
단순 해시 (SHA-256 등)	계산이 빨라서 흔한 비밀번호를 초당 수십억 개 대입해 찾아냄
bcrypt (소금 + 느린 계산)	한 번 계산에 약 0.2초. 회원마다 소금이 달라 미리 만든 표로 찾을 수 없음

3. 코드

import (파일 맨 위)

```
import re
import string
```

```
import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다. **`import bcrypt` 를 이번에 추가했고**, 이 함수는 그것만 쓴다. `bcrypt` 패키지는 `requirements.txt` 에 이미 있다 (설치된 버전 5.0.0).

함수

```
def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode("utf-8"), bcrypt.gensalt()).decode("ascii")
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와 `모듈.이름` 으로 씀 (예: `bcrypt.hashpw`)

한 줄을 안쪽부터 풀면 다음과 같다.

순서	식	값
①	`plain.encode("utf-8")`	`"Abcdef1!"` → `b"Abcdef1!"` (글자 → 바이트. bcrypt 는 바이트만 받음)
②	`bcrypt.gensalt()`	`b"$2b$12$Q9xKc3…"` — 새 소금 (기본 비용 12)
③	`bcrypt.hashpw(①, ②)`	`b"$2b$12$Q9xKc3…(60바이트)"` — 해시 (바이트)
④	`.decode("ascii")`	바이트 → 글자. DB 의 `VARCHAR` 칸에 넣으려고

식	뜻
`b"…"`	바이트 값. 글자(`str`)와 다른 타입
`"ascii"` 로 되돌림	bcrypt 해시는 영문 · 숫자 · `$./` 만 쓰므로 ASCII 로 충분

4. 흐름

1. 비밀번호를 바이트로 바꾼다.
2. 새 소금을 만든다.
3. 비밀번호와 소금으로 해시를 계산한다 (약 0.2초).
4. 글자로 바꿔 돌려준다.

5. 왜 이렇게

선택	이유
bcrypt	PRD FR-102 가 정한 방식. 오래 검증된 비밀번호 전용 해시
소금을 매번 새로	같은 비밀번호라도 회원마다 해시가 다르다. 한 사람의 비밀번호를 알아내도 같은 비밀번호를 쓰는 다른 회원이 드러나지 않는다
소금을 따로 저장하지 않음	해시 안(`$2b$12$` 뒤 22자)에 들어 있다. 칸 하나(`password_hash`)로 충분하다

선택	이유
비용 12 (기본값)	이 컴퓨터에서 한 번에 약 0.19초. 가입 · 로그인 한 번에는 느끼지 못하지만, 대입 공격에는 매우 느리다
72바이트 제한은 따로 처리하지 않음	bcrypt 5 는 72바이트가 넘으면 오류를 낸다. 비밀번호는 14 에서 최대 20자 · 영문 · 숫자 · 기호(1글자 = 1바이트)로 검사한 뒤 들어오므로 넘을 수 없다

6. 시험 (`tests/test_password.py` 에 5개 추가 · 모두 21개)

시험	확인한 것
모양	글자이고, `$2b$12$` 로 시작하고, 60자 (`VARCHAR(255)` 에 충분)
매번 다름	같은 비밀번호를 두 번 해시하면 결과가 다름 (소금)
원래 비밀번호만 맞음	`bcrypt.checkpw` 로 `"Abcdef1!"` 은 맞고 `"Abcdef1?"` 는 틀림
비밀번호가 보이지 않음	해시 안에 원래 글자가 없음
가장 긴 비밀번호 (20자)	오류 없이 해시되고 비교도 맞음

비교 시험은 다음 함수(16 `verify_password`) 대신 `bcrypt.checkpw` 를 직접 썼다.

전체 시험 97개 통과, `ruff` 검사 통과. 해시 한 번 약 190ms.

Backend(FastAPI/Python) 16 — `verify_password()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입 (템플릿은 P1-05 — 15 와 짝이라 함께 구현) · PRD 01 FR-103
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	15 `hash_password()` · 17 `normalize_email()`

1. 하는 일

입력한 비밀번호가 DB 에 저장된 해시(15)와 **맞는지** `True` / `False` 로 알려 준다.

호출	결과
`verify_password("Abcdef1!", "$2b$12$Q9xK…")` (가입 때 쓴 비밀번호)	`True`
`verify_password("Abcdef1?", "$2b$12$Q9xK…")`	`False`

해시를 원래 비밀번호로 **되돌리는 것이 아니다.** 해시 안에 있는 소금(15 §1)을 꺼내, 입력한 비밀번호를 같은 방법으로 다시 계산하고 결과가 같은지 본다.

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `authenticate()`	home.html 로그인 모달	이메일로 찾은 회원의 `password_hash` 와 비교. `False` 면 401 "Email or password is incorrect."
P1-09 `check_current_password()`	userInfo.html 비밀번호 변경 · 탈퇴	현재 비밀번호 확인. `False` 면 400 "Password is incorrect."

3. 코드

import (파일 맨 위)

```
import re
import string

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다 (15 와 같음, 이번에 추가한 것 없음). 이 함수는 **`bcrypt`** 만 쓴다.

함수

```
def verify_password(plain: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("ascii"))
    except ValueError:
        return False
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와 `모듈.이름` 으로 씀 (예: `bcrypt.checkpw`)

식	뜻
`bcrypt.checkpw(비밀번호, 해시)`	해시 안의 소금으로 비밀번호를 다시 계산해 같으면 `True`. 둘 다 바이트로 넘김
`plain.encode("utf-8")` · `hashed.encode("ascii")`	글자 → 바이트 (15 의 반대 방향)
`try: … except ValueError: return False`	`ValueError` 가 나면 오류로 멈추지 않고 `False`

입력	`bcrypt.checkpw`	이 함수
맞는 비밀번호	`True`	`True`
틀린 비밀번호	`False`	`False`
72바이트가 넘는 비밀번호 (`"A" * 100`)	**`ValueError`**	`False`
망가진 해시 (`""`, `"x"`)	**`ValueError`** ("Invalid salt")	`False`

4. 흐름

1. 비밀번호와 해시를 바이트로 바꾼다.
2. bcrypt 가 해시에서 소금 · 비용을 꺼내 비밀번호를 다시 계산한다 (약 0.2초).
3. 결과가 같으면 `True`, 다르면 `False`.
4. 계산할 수 없는 입력이면 `False`.

5. 왜 이렇게

선택	이유
`ValueError` 를 `False` 로	로그인 입력은 100자까지 받는다(`LoginRequest`). 비밀번호 규칙(14)은 가입 때만 검사하므로, 로그인 칸에 긴 글을 넣으면 bcrypt 가 오류를 던져 **500** 이 난다. 그런 비밀번호로 가입한 회원은 없으니 "틀림"이 맞는 답이다
망가진 해시도 `False`	DB 에 잘못된 값이 있어도 로그인이 500 으로 깨지지 않고 "틀림"으로 끝난다
`==` 로 직접 비교하지 않음	`bcrypt.checkpw` 는 비교에 걸리는 시간이 결과와 상관없이 같다. 시간 차이로 해시를 추측하는 공격을 막는다
템플릿(P1-05)보다 먼저 구현	15 와 짝이라, 15 의 시험을 "해시 → 비교" 로 함께 할 수 있다

P1-05 에서 알아 둘 것: 이메일이 없는 회원이면 이 함수를 부르지 않아 응답이 0.2초 빨라진다. 그 차이로 "이 이메일은 가입되어 있다"는 것을 알아낼 수 있다. 그래서 `authenticate()` 템플릿에는 회원이 없을 때도 가짜 해시로 이 함수를 한 번 부르라고 적혀 있다.

6. 시험 (`tests/test_password.py` 에 11개 추가 · 모두 32개)

시험	확인한 것
맞는 비밀번호	`True`
틀린 비밀번호 5개	한 글자 다름 · 대소문자 · 뒤 공백 · 빈 글자 · 두 번 반복 → `False`
너무 긴 비밀번호	`"A" * 100` · 한글 30자(90바이트) → 오류 없이 `False`
망가진 해시 4개	`""` · `"x"` · `"not-a-bcrypt-hash"` · `"$2b$12$짧음"` → 오류 없이 `False`

전체 시험 108개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 17 — `normalize_email()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입 — 이메일 비교 기준 · PRD 01 FR-101
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	16 `verify_password()` · 18 `release_withdrawn()`

1. 하는 일

이메일의 **앞뒤 공백을 지우고 소문자로** 바꾼다. 같은 사람이 다르게 입력한 이메일을 하나의 값으로 맞춘다.

입력	결과
`"User@Example.COM"`	`"user@example.com"`
`" user@example.com "`	`"user@example.com"`
`"first.last+tag@Mail.Example.com"`	`"first.last+tag@mail.example.com"` (점 · `+` 는 그대로)

2. 언제 쓰나

이메일을 **DB 와 비교하거나 저장하기 전에 항상** 거친다.

쓰는 곳	하는 일
19 `check_email()`	join.html 이메일 칸 실시간 중복 확인
21 `signup()`	가입할 때 중복 확인 + 저장
P1-05 `authenticate()`	로그인 모달에서 입력한 이메일로 회원 찾기

없으면 생기는 문제

순서	일어나는 일
①	A 가 `kim@naver.com` 으로 가입
②	B 가 `Kim@Naver.com` 으로 가입 → DB 의 `UNIQUE` 는 글자가 다르니 통과 (MySQL 설정에 따라 다름)
③	두 계정이 실제로는 **같은 메일함**을 가리킴. 비밀번호 찾기 메일이 엉뚱한 사람에게 갈 수 있음
④	또는 A 가 로그인할 때 `Kim@naver.com` 이라고 치면 회원을 못 찾음

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다. **이 함수는 import 한 것을 쓰지 않는다** — `strip()` · `lower()` 는 파이썬 글자(`str`)에 기본으로 있는 기능이다.

함수

```
def normalize_email(email: str) -> str:
    return email.strip().lower()
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻	예
`글자.strip()`	앞뒤의 공백 · 탭 · 줄바꿈을 지움. **가운데는 그대로**	`" a b "` → `"a b"`
`글자.lower()`	모두 소문자로	`"Kim@Naver.COM"` → `"kim@naver.com"`
`.strip().lower()`	앞 결과에 이어서 다음 기능을 부름 (왼쪽부터 차례대로)	

4. 흐름

1. 앞뒤 공백을 지운다.
2. 소문자로 바꾼다.

5. 왜 이렇게

선택	이유
함수 하나로 모음	중복 확인 · 가입 · 로그인이 **같은 기준**으로 비교해야 한다. 한 곳이라도 빠지면 위 §2 의 문제가 생긴다
앞부분(`@` 앞)까지 소문자로	규격상 `@` 앞은 대소문자를 구분할 수 있지만, 네이버 · Gmail 등 실제 메일 서비스는 구분하지 않는다. 사람이 대소문자를 섞어 쳐도 같은 계정이 되는 편이 안전하다
점 · `+` 는 건드리지 않음	Gmail 은 `a.b@` 와 `ab@` 를 같게 보지만, 다른 서비스는 아니다. 서비스마다 규칙이 달라 손대지 않는다
가운데 공백은 지우지 않음	`"a b@x.com"` 은 틀린 형식이다. 고쳐 주지 않고 입력 검사(`EmailStr`, 25)가 422 로 돌려준다
화면에서 보낸 값은 그대로 두고 여기서 맞춤	브라우저가 무엇을 보내든 서버가 마지막으로 같은 기준을 지킨다

6. 시험 (`tests/test_auth_service.py`, 새로 만듦 · 7개)

시험	확인한 것
5가지 입력	그대로 · 대문자 · 앞뒤 공백 · 탭과 줄바꿈 · 점과 `+` 유지 → 위 표의 결과
같은 사람의 다른 입력	`" Kim@Naver.com"` 과 `"kim@naver.com "` 이 같은 값이 됨
가운데 공백	지우지 않음 (`"a b@x.com"`)

전체 시험 115개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 18 — `release_withdrawn()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입 — 탈퇴 후 재가입 · PRD 01 FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	17 `normalize_email()` · 19 `check_email()`

1. 하는 일

탈퇴한 회원의 이메일 · 닉네임을 **다른 값으로 바꿔서**, 같은 이메일 · 닉네임으로 새로 가입할 수 있게 한다.

칸	바꾸기 전 (탈퇴한 85번 회원)	바꾼 뒤
`email`	`kim@naver.com`	`deleted+85@withdrawn.invalid`
`nickname`	`beatmaker`	`deleted_user_0000000085`
`status` · `deleted_at` · 프로젝트		**그대로**

왜 필요한가

순서	일어나는 일
①	85번 회원이 탈퇴 → 행은 지우지 않고 `status=DELETED` (P1-09, 소프트 삭제)
②	행이 `kim@naver.com` · `beatmaker` 를 그대로 쥐고 있음
③	4일 뒤 같은 사람이 `kim@naver.com` 으로 다시 가입 → `email` 은 `UNIQUE` 라 DB 가 거절
④	그래서 ③ 직전에 이 함수가 85번 행의 값을 비켜 둠 → 새 행이 들어갈 수 있음

2. 언제 쓰나

"그 값으로 다시 가입하려는 사람이 나타났을 때" 부른다. 탈퇴 3일이 지났는지는 **부르는 쪽이 확인**한다.

쓰는 곳	화면	상황
19 `check_email()` · 20 `check_nickname()`	join.html 중복 확인	그 값을 쥔 행이 탈퇴 후 3일이 지났으면 비켜 두고 "쓸 수 있음"
21 `signup()`	join.html Sign Up 버튼	가입 직전 같은 확인

탈퇴 후	결과 (FR-108, 2026-10-03 결정)
3일 안	비켜 두지 않음 → "You can sign up again after …"
3일 지남	**이 함수** → 새 가입 가능

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다. 이 함수는 **`Session`** · **`User`** 를 타입 표시에 쓴다.

함수

```
def release_withdrawn(db: Session, user: User) -> None:
    user.email = f"deleted+{user.user_id}@withdrawn.invalid"
    user.nickname = f"deleted_user_{user.user_id:010d}"
    db.flush()
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻	85번 회원일 때
`f"deleted+{user.user_id}@…"`	f-문자열. `{ }` 안의 값을 글자에 끼움	`deleted+85@withdrawn.invalid`
`{user.user_id:010d}`	정수(`d`)를 **10자리**로, 빈자리는 `0` 으로 채움	`0000000085`
`db.flush()`	바꾼 값을 지금 DB 에 보냄 (`UPDATE`). 아직 확정(`commit`)은 아님	
`-> None`	돌려주는 값이 없음	

4. 흐름

1. 이메일을 `deleted+<번호>@withdrawn.invalid` 로 바꾼다.
2. 닉네임을 `deleted_user_<번호 10자리>` 로 바꾼다.
3. `flush` 로 DB 에 먼저 반영한다 → 같은 저장 안에서 이어 넣는 새 행이 `UNIQUE` 에 걸리지 않는다.

5. 왜 이렇게

선택	이유
행을 지우지 않고 값만 바꿈	탈퇴 회원이 남긴 프로젝트(`owner_id`) · 문의 기록이 그 행을 가리킨다. 지우면 FK 때문에 지울 수도 없다
회원 번호를 넣음	탈퇴 회원마다 값이 달라서 비켜 둔 값끼리도 `UNIQUE` 에 걸리지 않는다
이메일 도메인 `.invalid`	인터넷 규격(RFC 2606)상 **실제로 존재할 수 없는** 도메인이다. 이 주소로 메일이 갈 일이 없다
닉네임을 템플릿(`deleted_85`)과 다르게 **23자**로	`deleted_85` 는 회원이 직접 가입할 수 있는 닉네임(2 ~ 20자)이다. 누가 먼저 그 이름을 쓰고 있으면 비켜 둘 때 `UNIQUE` 충돌 → 500. 가입 닉네임은 최대 20자라 21자 이상인 이름은 **절대 겹치지 않는다** (칸은 50자까지)
`flush` (`commit` 아님)	가입과 한 저장으로 묶는다. 가입이 실패하면 비켜 둔 것도 함께 취소된다

선택	이유
3일 확인은 부르는 쪽에서	이 함수는 "비켜 두기" 한 가지만 한다. 언제 비켜 둘지는 19 ~ 21 이 정한다

탈퇴 회원 추적 방지 (2026-10-07 결정)

비켜 둔 값에는 **회원 번호만** 들어 있다. 원래 이메일 · 닉네임은 덮어써져 남지 않으므로, 이 값으로 그 사람이 누구였는지 알아낼 수는 없다. 회원 번호 자체도 바깥에 드러나지 않게 아래를 지킨다.

대책	언제 · 어디서	막는 것
① 다른 사람에게는 **'Deleted user'** 로만 보냄. 그 사람의 `user_id` 도 응답에 넣지 않음	Phase 6 멤버 목록 · 채팅 · 댓글 (`user_service.withdraw` TODO 에 기록)	`deleted_user_0000000085` 가 화면에 보여 회원 번호가 드러나는 것
② 3일 뒤 남은 개인 정보도 지움: `password_hash = "!"` · `last_login_at` · `profile_image_path` 비움	보관 기간 정리 `retention.py` (P1-09, TODO 에 기록)	탈퇴한 행에 개인 정보가 계속 남는 것
③ 회원 번호로 다른 회원을 찾는 공개 API 를 만들지 않음	지금 API 확인함 — 회원 번호는 가입 응답 · 내 정보(본인)에만 나감	번호를 알아도 그 번호로 아무것도 조회할 수 없게

DB 를 직접 보는 관리자(SQLAdmin, P1-21)는 회원 번호와 탈퇴 행을 볼 수 있다. 관리 목적상 필요하고, 관리자 화면은 ADMIN 계정만 들어간다.

6. 시험 (`tests/test_auth_service.py` 에 4개 추가 · 모두 11개, 실제 MySQL · rollback)

시험	확인한 것
비켜 둔 뒤 같은 값으로 가입	이메일 · 닉네임이 바뀌고, 같은 `gone@example.com` · `gone_nick` 으로 새 회원이 들어감
비켜 두지 않으면	같은 이메일로 넣을 때 DB 가 거절 (`IntegrityError`) — 이 함수가 필요한 이유
행 · 프로젝트는 그대로	`status=DELETED` · `deleted_at` 그대로, 남긴 프로젝트의 `owner_id` 그대로
닉네임 충돌 없음	비켜 둔 닉네임이 20자보다 길고 50자 이하. 회원이 `deleted_85` 같은 닉네임을 써도 충돌 없음

전체 시험 119개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.

Backend(FastAPI/Python) 19 — `check_email()` · `availability_of()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입 — 이메일 중복 확인 · 탈퇴 후 3일 규칙 · PRD 01 FR-101 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	18 `release_withdrawn()` · 20 `check_nickname()`

1. 하는 일

이 이메일로 **새로 가입할 수 있는지** 알려 준다.

DB 에 그 이메일을 가진 행	결과	join.html 이메일 칸 아래 문구
없음	`{"available": true}`	✓ Available
활동 중(`ACTIVE`) · 정지(`SUSPENDED`) 회원	`{"available": false, "reason": "TAKEN"}`	This email is already in use.
탈퇴 후 **3일 안**	`{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "2026-10-09T03:00:00Z"}`	You can sign up again after Oct 9.
탈퇴 후 3일 지남	`{"available": true}` (가입 때는 18 로 비켜 둠)	✓ Available

함수	역할
`check_email()`	이메일로 행을 찾음
`availability_of()`	찾은 행을 보고 위 표대로 판단. 20 `check_nickname()` 도 같은 판단을 씀

2. 언제 쓰나

부르는 곳	`release`	하는 일
`GET /auth/check-email` (24) — join.html 이메일 칸의 **Check** 버튼	`False`	확인만. DB 를 바꾸지 않음
21 `signup()` — **Sign Up** 버튼	`True`	3일 지난 탈퇴 행이면 18 로 비켜 둔 뒤 가입

3일 규칙이 있는 이유 (FR-108, 2026-10-03 결정): 탈퇴 직후 다른 사람이 같은 이메일 · 닉네임으로 바로 가입하면, 탈퇴한 사람인 척할 수 있다. 3일 동안은 그 값을 쓰지 못하게 막는다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.orm import Session
```

```
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다. **`datetime` 줄 · `select` · `UserStatus` 를 이번에 추가했다.** 이 두 함수는 `SignupRequest` 를 뺀 나머지를 모두 쓴다.

import	가져오는 것	쓰는 곳
`UTC` · `datetime` · `timedelta` (파이썬 기본)	시간대 · 시각 · 기간	지금 시각, 탈퇴 + 3일
`select`	SQL `SELECT` 를 파이썬으로	이메일로 행 찾기
`UserStatus`	08 의 회원 상태 목록	`DELETED` 인지
`AvailabilityResponse`	결과 모양 (`schemas/auth.py`)	돌려주는 값

함수

```
REJOIN_WAIT_DAYS = 3


def check_email(db: Session, email: str, release: bool = False) -> AvailabilityResponse:
    user = db.scalar(select(User).where(User.email == normalize_email(email)))
    return availability_of(db, user, release)


def availability_of(db: Session, user: User | None, release: bool) -> AvailabilityResponse:
    if user is None:
        return AvailabilityResponse(available=True)
    if user.status != UserStatus.DELETED:
        return AvailabilityResponse(available=False, reason="TAKEN")
    if user.deleted_at is not None:
        available_at = user.deleted_at + timedelta(days=REJOIN_WAIT_DAYS)
        now = datetime.now(UTC).replace(tzinfo=None)  # DB 의 시각은 UTC · 시간대 정보 없음
        if now < available_at:
            return AvailabilityResponse(
                available=False,
                reason="REJOIN_TOO_SOON",
                available_at=available_at.replace(tzinfo=UTC),
            )
    if release:
        release_withdrawn(db, user)
    return AvailabilityResponse(available=True)
```

문법 풀이

import 문법	뜻
`from 모듈 import A, B, C`	한 모듈에서 여러 이름을 한 줄로 가져옴

식	뜻
`release: bool = False`	기본값이 있는 인자. 안 넘기면 `False` (중복 확인 API), 가입은 `release=True` 로 부름
`select(User).where(User.email == …)`	`SELECT * FROM users WHERE email = …`
`db.scalar(…)`	결과 **한 행**을 `User` 로, 없으면 `None` (`email` 은 `UNIQUE` 라 많아야 1행)

식	뜻
`User | None`	회원 또는 `None`
`timedelta(days=3)`	3일이라는 기간. 시각에 더하면 3일 뒤 시각
`datetime.now(UTC).replace(tzinfo=None)`	지금 UTC 시각에서 시간대 표시를 뗌 — DB 에서 읽은 `deleted_at` 도 표시가 없어서, 둘을 비교하려면 모양을 맞춰야 함
`available_at.replace(tzinfo=UTC)`	화면에 보낼 때는 UTC 표시를 붙임 → JSON 에 `…Z` 로 나감. 화면이 사용자 시간대로 바꿔 보여 줌

4. 흐름

1. 이메일을 17 로 맞춘 뒤 그 이메일을 가진 행을 찾는다.
2. 없으면 → 쓸 수 있음.
3. 탈퇴가 아니면(활동 · 정지) → `TAKEN`.
4. 탈퇴했고 3일이 안 지났으면 → `REJOIN_TOO_SOON` + 언제부터 되는지.
5. 3일이 지났으면 → (`release` 면 18 로 비켜 두고) 쓸 수 있음.

5. 왜 이렇게

선택	이유
`release` 인자 추가 (템플릿에 없음)	중복 확인은 `GET` 요청이다. `GET` 은 "보기만" 하는 요청이라 DB 를 바꾸면 안 된다 (새로 고침 · 미리 불러오기로 여러 번 불릴 수 있음). 실제로 비켜 두는 것은 가입할 때만 한다
판단을 `availability_of()` 로 분리	닉네임(20)도 규칙이 같다. 한 곳에서만 고치면 둘 다 바뀐다
정지(`SUSPENDED`) 회원도 `TAKEN`	정지는 탈퇴가 아니다. 계정이 살아 있으니 이메일도 그 사람 것이다
`deleted_at` 이 비어 있으면 3일이 지난 것으로 봄	정상이라면 탈퇴 때 항상 기록된다(P1-09). 값이 없는 이상한 행 때문에 그 이메일이 영원히 막히지 않게 한다
`available_at` 을 UTC 로 보냄	서버는 시각을 모두 UTC 로 다룬다(`docs/DB.md`). 한국 · 미국 사용자 모두 자기 시간대로 볼 수 있다
이유를 `TAKEN` · `REJOIN_TOO_SOON` 으로 나눔	화면이 다른 문구를 보여 줄 수 있다. "이미 사용 중" 과 "며칠 뒤 가능" 은 사용자가 할 일이 다르다

참고: 이 확인은 "그 이메일이 가입되어 있는지"를 누구에게나 알려 준다(이메일 존재 확인). 실시간 중복 확인 기능(FR-101)을 만들면 피할 수 없는 성질이라, 대부분의 사이트도 같은 방식이다. 마구 묻는 것을 막는 요청 횟수 제한은 P1-12 의 제한 방식을 함께 쓰는 것을 그때 검토한다.

6. 시험 (`tests/test_auth_service.py` 에 8개 추가 · 모두 19개, 실제 MySQL · rollback)

시험	확인한 것
없는 이메일	`available=True`
활동 · 정지 회원 (2개)	`TAKEN`
대소문자 · 공백 섞어 물음	`" Taken@Example.COM "` 도 `TAKEN` (17 로 맞춘 뒤 비교)
탈퇴 2일	`REJOIN_TOO_SOON`, `available_at` = 탈퇴 + 3일, UTC 표시 붙음

시험	확인한 것
탈퇴 3일 1시간 · 확인만	`available=True`, 이메일은 **그대로**
탈퇴 4일 · `release=True`	`available=True`, 이메일이 `deleted+…` 로 바뀌고 같은 이메일로 바로 가입됨
응답 모양	`{"available": true, "reason": null, "available_at": null}`

전체 시험 127개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.

Backend(FastAPI/Python) 20 — `check_nickname()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입 — 닉네임 중복 확인 · PRD 01 FR-101 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	19 `check_email()` · 21 `signup()`

1. 하는 일

이 닉네임을 **새로 쓸 수 있는지** 알려 준다. 판단 규칙은 19 와 같다 (`availability_of()` 를 함께 씀).

DB 에 그 닉네임을 가진 행	결과
없음	`{"available": true}`
활동 · 정지 회원	`{"available": false, "reason": "TAKEN"}`
탈퇴 후 3일 안	`{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "…Z"}`
탈퇴 후 3일 지남	`{"available": true}` (가입 때는 18 로 비켜 둠)

대소문자가 달라도 같은 닉네임

이미 있는 닉네임	확인하는 닉네임	결과
`beatmaker`	`beatmaker`	`TAKEN`
`beatmaker`	`BeatMaker` · `BEATMAKER`	`TAKEN`
`beatmaker`	`" beatmaker "` (앞뒤 공백)	`TAKEN`
`café`	`cafe`	`TAKEN` (악센트도 구분 안 함)

2. 언제 쓰나

부르는 곳	`release`	화면
`GET /auth/check-nickname` (24)	`False`	join.html 닉네임 칸의 **Check** 버튼
21 `signup()`	`True`	**Sign Up** 버튼
P1-08 `update_me()`	`False` 또는 `True`	userInfo.html 닉네임 바꾸기 (그때 정함)

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.orm import Session
```

```
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다 (19 와 같음, 이번에 추가한 것 없음). 이 함수는 **`select`** · **`Session`** · **`User`** · **`AvailabilityResponse`** 를 쓰고, 판단은 19 의 `availability_of()` 에 맡긴다.

함수

```
def check_nickname(db: Session, nickname: str, release: bool = False) -> AvailabilityResponse:
    user = db.scalar(select(User).where(User.nickname == nickname.strip()))
    return availability_of(db, user, release)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻
`nickname.strip()`	앞뒤 공백만 지움. **대소문자는 바꾸지 않음** (이메일과 다름)
`User.nickname == …`	SQL `WHERE nickname = …`. 실제 비교는 **MySQL 이 함** — 아래 정렬 규칙을 따름

MySQL 의 비교 규칙 (정렬 규칙 `utf8mb4_0900_ai_ci`)

`users.nickname` · `users.email` 칸은 이 규칙을 쓴다. 이 DB 에서 직접 확인한 결과다.

비교	결과	뜻
`'Beat' = 'beat'`	같음	`ci` = case-insensitive (대소문자 구분 안 함)
`'café' = 'cafe'`	같음	`ai` = accent-insensitive (악센트 구분 안 함)
`'ａ' = 'a'`	같음	전각 글자도 같게 봄
`'비트' = '비트 '`	**다름**	뒤 공백을 그대로 비교 (`NO PAD`) → 그래서 `strip()` 이 필요

4. 흐름

1. 앞뒤 공백을 지운다.
2. 그 닉네임을 가진 행을 DB 에서 찾는다 (대소문자 · 악센트 무시).
3. 19 의 `availability_of()` 로 판단한다.

5. 왜 이렇게

선택	이유
닉네임은 소문자로 바꾸지 않음	닉네임은 화면에 보이는 이름이다. `BeatMaker` 로 가입한 사람의 표기를 그대로 지킨다
그래도 `BeatMaker` 와 `beatmaker` 는 같은 이름	DB 의 `UNIQUE` 가 이미 그렇게 비교한다. 다르게 확인하면 "✓ Available" 이라고 해 놓고 가입 때 DB 가 거절하는 일이 생긴다. 또 `beatmaker` 를 흉내 낸 `BeatMaker` 계정(사칭)을 막는 효과도 있다

선택	이유
비교를 파이썬이 아니라 DB 에 맡김	확인 결과와 가입 때 DB 판단이 늘 같다
앞뒤 공백은 직접 지움	MySQL 이 뒤 공백을 다르게 본다(`NO PAD`). `"beatmaker "` 로 따로 가입되면 화면에서는 같아 보이는 두 이름이 생긴다
판단은 19 와 같은 함수	탈퇴 3일 규칙 · 정지 회원 처리가 이메일과 똑같아야 한다

6. 시험 (`tests/test_auth_service.py` 에 8개 추가 · 모두 27개, 실제 MySQL · rollback)

시험	확인한 것
없는 닉네임	`available=True`
같은 닉네임	`TAKEN`
공백 · 대소문자 (3개)	`" beatmaker "` · `BeatMaker` · `BEATMAKER` 모두 `TAKEN`
DB 도 같은 규칙인지	`beatmaker` 가 있을 때 `BeatMaker` 로 넣으면 DB 가 거절 (`IntegrityError`) — 확인 결과와 일치
탈퇴 1일	`REJOIN_TOO_SOON`
탈퇴 5일	확인만 하면 그대로, `release=True` 면 `deleted_user_…` 로 비켜 두고 같은 닉네임으로 가입됨

전체 시험 135개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.

Backend(FastAPI/Python) 21 — `signup()` · `unavailable_error()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입 — 가입 처리 · PRD 01 FR-101 · FR-102 · FR-108 · 처리방침 동의
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	20 `check_nickname()` · 22 `POST /auth/signup`

1. 하는 일

join.html 의 **Sign Up** 버튼으로 온 가입 요청을 검사하고, 통과하면 `users` 에 새 회원을 저장한다. 14 ~ 20 을 모두 엮는 함수다.

함수	역할
`signup()`	검사 → 저장 → 저장된 `User` 를 돌려줌
`unavailable_error()`	19 · 20 의 "쓸 수 없음" 결과를 409 오류로 바꿈

경우	결과
모두 통과	새 `User` (번호 · 가입 시각은 DB 가 채움)
비밀번호 규칙 위반	422 `VALIDATION_ERROR` "Check the password requirements." + `details.password: ["upper", …]`
처리방침 동의 안 함	422 "Please agree to the Privacy Policy."
예전 처리방침 화면에서 보냄	422 "Please reload the page."
이메일 사용 중	409 `EMAIL_TAKEN` "This email is already in use."
닉네임 사용 중	409 `NICKNAME_TAKEN` "This nickname is already in use."
탈퇴 후 3일 안	409 `REJOIN_TOO_SOON` "You can sign up with this email again after Oct 9, 2026 (UTC)."

2. 언제 쓰나

`POST /api/v1/auth/signup` (22) 이 이 함수를 부른다. 화면 흐름은 다음과 같다.

순서	화면 · 서버	일어나는 일
①	join.html	이메일 · 닉네임 **Check** 버튼 (19 · 20) → ✓ Available
②	join.html	비밀번호 조건 목록이 모두 ✓, "만 16세 이상 + 처리방침 동의" 체크
③	join.js (P1-07)	**Sign Up** → `POST /auth/signup` `{email, nickname, password, privacy_agreed, privacy_version}`
④	**이 함수**	같은 검사를 서버에서 한 번 더 → 저장
⑤	join.js	201 → 홈으로 이동해 로그인 모달 (P1-07)

①② 를 화면에서 했어도 ④ 에서 **다시 검사**한다. 화면 검사는 건너뛸 수 있고, ① 과 ③ 사이에 다른 사람이 그 이메일로 먼저 가입할 수도 있다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import check_password_rules, hash_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다. **`IntegrityError` · `AppError` · `ErrorCode` · `check_password_rules` · `hash_password` · `PRIVACY_VERSION` 을 이번에 추가했다.**

import	가져오는 것	쓰는 곳
`IntegrityError`	DB 가 `UNIQUE` 등을 어겨 거절할 때의 오류	동시 가입 처리
`AppError` · `ErrorCode`	01 의 오류	422 · 409
`check_password_rules` · `hash_password`	14 · 15	비밀번호 검사 · 해시
`PRIVACY_VERSION`	지금 처리방침 버전 `"2026-10-05"` (`schemas/auth.py`)	동의한 판 확인
`SignupRequest`	가입 요청 모양	`data`

함수

```
def signup(db: Session, data: SignupRequest) -> User:
    failed = check_password_rules(data.password)
    if failed:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Check the password requirements.",
                       details={"password": failed})
    if not data.privacy_agreed:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Please agree to the Privacy Policy.",
                       details={"privacy_agreed": "required"})
    if data.privacy_version != PRIVACY_VERSION:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Please reload the page.",
                       details={"privacy_version": "outdated"})

    email = normalize_email(data.email)
    nickname = data.nickname.strip()
    email_result = check_email(db, email, release=True)
    if not email_result.available:
        raise unavailable_error("email", email_result)
    nickname_result = check_nickname(db, nickname, release=True)
    if not nickname_result.available:
        raise unavailable_error("nickname", nickname_result)

    user = User(
        email=email,
        nickname=nickname,
```

```
        password_hash=hash_password(data.password),
        privacy_agreed_at=datetime.now(UTC).replace(tzinfo=None),
        privacy_version=PRIVACY_VERSION,
    )
    db.add(user)
    try:
        db.commit()
    except IntegrityError as exc:
        # 동시에 같은 값으로 가입한 사람이 먼저 저장된 경우 (위 검사는 둘 다 통과)
        db.rollback()
        if "uq_users_nickname" in str(exc.orig):
            field = "nickname"
        else:
            field = "email"
        raise unavailable_error(field, AvailabilityResponse(available=False, reason="TAKEN")) from None
    db.refresh(user)
    return user


def unavailable_error(field: str, result: AvailabilityResponse) -> AppError:
    if result.reason == "REJOIN_TOO_SOON":
        when = result.available_at
        day = f"{when:%b} {when.day}, {when.year}"  # 예: Oct 9, 2026
        return AppError(ErrorCode.REJOIN_TOO_SOON, 409,
                        f"You can sign up with this {field} again after {day} (UTC).",
                        details={field: "REJOIN_TOO_SOON", "available_at": when.isoformat()})
    if field == "email":
        return AppError(ErrorCode.EMAIL_TAKEN, 409, "This email is already in use.")
    return AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.")
```

(실제 파일은 `ruff` 가 `AppError(...)` 인자를 한 줄에 하나씩 나눠 적었다. 내용은 같다.)

문법 풀이

import 문법	뜻
`from sqlalchemy.exc import IntegrityError`	라이브러리 안 `exc`(오류 모음) 모듈에서 오류 종류 하나를 가져옴

식	뜻
`if failed:`	목록이 비어 있으면 거짓, 하나라도 있으면 참
`raise unavailable_error(…)`	함수가 **만들어 돌려준** 오류를 던짐 (`unavailable_error` 는 던지지 않고 돌려주기만 함)
`db.add(user)` → `db.commit()`	저장 대기 → 확정. 확정할 때 DB 가 `UNIQUE` 를 마지막으로 검사
`except IntegrityError as exc:`	DB 가 거절하면 그 오류를 `exc` 로 받음
`str(exc.orig)`	MySQL 이 보낸 원래 오류 글. `Duplicate entry 'newbie' for key 'users.uq_users_nickname'` 처럼 어느 제약인지 들어 있음
`db.rollback()`	실패한 저장을 취소. 하지 않으면 세션을 더 쓸 수 없음
`raise … from None`	DB 오류를 이어 붙이지 않고 우리 오류만 던짐 (05 와 같음)
`db.refresh(user)`	DB 가 채운 값(`user_id` · `created_at` · `role` · `status` …)을 다시 읽음
`f"{when:%b}"`	날짜를 영어 월 약자로: `Oct`

식	뜻
`when.isoformat()`	`"2026-10-09T03:00:00+00:00"` — 화면(JS)이 읽기 쉬운 표준 모양

4. 흐름

1. 비밀번호 규칙(14) → 처리방침 동의 → 처리방침 판 확인. **DB 를 보지 않는** 검사를 먼저 한다.
2. 이메일(19) → 닉네임(20) 확인. 3일 지난 탈퇴 행이면 비켜 둔다(18).
3. 새 `User` 를 만든다 — 이메일은 17 로 맞추고, 비밀번호는 15 로 해시하고, 동의 시각을 지금(UTC)으로 적는다.
4. 저장(`commit`). DB 가 거절하면(동시 가입) 409.
5. DB 가 채운 값을 읽어 돌려준다.

5. 왜 이렇게

선택	이유
DB 를 보지 않는 검사를 먼저	틀린 요청은 DB 에 묻기 전에 끝낸다. 비켜 두기(18)도 다른 검사를 모두 통과한 뒤에만 일어난다
중간에 실패하면 `commit` 하지 않음	이메일을 비켜 둔 뒤 닉네임이 막혀도, 요청이 끝날 때 세션이 닫히며 비켜 둔 것까지 취소된다
`role` · `status` · 용량은 넣지 않음	DB 기본값(`USER` · `ACTIVE` · 5GB)이 정한다 (08 과 같은 원칙)
`privacy_agreed_at` 을 서버 시각으로	화면이 보낸 시각은 믿을 수 없다. 동의 기록은 서버가 남긴다 (처리방침 D3 · D4)
`privacy_version` 확인	처리방침이 바뀐 뒤 예전 화면에서 보낸 가입이면, 새 판을 보고 다시 동의하게 한다
`IntegrityError` → 409	① 확인과 ③ 저장 사이에 다른 사람이 먼저 저장하면 확인은 둘 다 통과한다. DB 의 `UNIQUE` 가 마지막에 막고, 그것을 500 이 아니라 "이미 사용 중" 으로 바꾼다
날짜에 "(UTC)" 표시	서버는 사용자 시간대를 모른다. 화면은 `details.available_at` 으로 사용자 시간대 날짜를 따로 보여 줄 수 있다
응답에 비밀번호 · 해시를 넣지 않음	이 함수는 `User` 를 돌려주지만, 라우터(22)는 `SignupResponse`(번호 · 이메일 · 닉네임)만 내보낸다

참고: 닉네임 2 ~ 20자 검사는 앞뒤 공백을 지우기 **전** 길이로 한다(스키마). `" a "` 는 3자로 통과한 뒤 `"a"`(1자)가 된다. 25(입력 스키마)에서 공백을 먼저 지우고 길이를 재도록 고친다.

6. 시험 (`tests/test_auth_service.py` 에 10개 추가 · 모두 37개, 실제 MySQL)

시험	확인한 것
가입 성공	이메일 소문자 · 닉네임 공백 제거, 해시 저장(원래 비밀번호 아님, 16 으로 맞음), `USER` · `ACTIVE`, 동의 판 · 시각 기록, DB 에 있음
비밀번호 규칙	422, `details.password = ["upper", "number", "special"]`
동의 안 함 · 예전 판	422 + 각 문구
이메일 사용 중	409 `EMAIL_TAKEN`
닉네임 대소문자만 다름	409 `NICKNAME_TAKEN`
탈퇴 1일	409 `REJOIN_TOO_SOON`, 문구에 "Oct 9, 2026 (UTC)" 모양 날짜, `details.available_at`

시험	확인한 것
탈퇴 4일	같은 이메일 · 닉네임으로 가입 성공, 예전 행은 `deleted+…` · `deleted_user_…`
동시 가입 (이메일 · 닉네임 2개)	확인을 통과한 것처럼 꾸민 뒤 저장 → DB 가 막음 → 500 이 아니라 409

시험 세션 변경 (`tests/conftest.py`): 이 함수는 `commit` 을 한다. 시험의 `db` 세션을 "바깥 트랜잭션 + 저장점(SAVEPOINT)" 방식으로 바꿔, 코드 안의 `commit` 은 저장점만 확정하고 시험이 끝나면 바깥을 통째로 되돌리게 했다. 앞의 모델 · 서비스 시험도 모두 이 방식으로 통과한다.

전체 시험 145개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.

Backend(FastAPI/Python) 22 — 라우터 `POST /api/v1/auth/signup`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `signup`)
작업	P1-04 회원가입 — 가입 API · PRD 01 FR-101 · §7
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	21 `signup()` · 23 `GET /auth/check-email`

1. 하는 일

브라우저가 보낸 가입 요청을 받아 21 `auth_service.signup()` 에 넘기고, 결과를 **응답 모양**으로 바꿔 돌려준다.

```
POST /api/v1/auth/signup
{"email": "Route.User@Example.com", "nickname": "route_user", "password": "Abcdef1!",
 "privacy_agreed": true, "privacy_version": "2026-10-05"}

→ 201 Created
{"user_id": 92, "email": "route.user@example.com", "nickname": "route_user"}
```

2. 언제 쓰나

join.html 의 **Sign Up** 버튼 → `join.js`(P1-07) 가 이 주소로 보낸다. 201 을 받으면 화면은 `home.html#login` 으로 옮겨 로그인 모달을 연다(가입 후 자동 로그인은 하지 않음).

라우터 · 서비스는 무엇이 다른가

층	파일	하는 일
라우터 (이 함수)	`api/v1/auth.py`	주소 · 방식(`POST`) · 응답 상태(201) · 응답 모양을 정함. HTTP 를 아는 층
서비스 (21)	`services/auth_service.py`	실제 검사 · 저장. HTTP 를 모름 — 시험이나 다른 곳에서도 그대로 부를 수 있음

요청 하나가 지나가는 길

순서	누가	일어나는 일
①	FastAPI	본문 JSON 을 `SignupRequest` 와 비교. 칸이 빠졌거나 모양이 틀리면 **라우터 실행 전에** 422 (02 처리기)
②	FastAPI	`db: Db` → `get_db()` 로 DB 세션을 엶
③	**이 함수**	`auth_service.signup(db, data)`
④	21	검사 · 저장. 실패하면 `AppError` → 01 처리기가 409 · 422 응답
⑤	**이 함수**	`User` → `SignupResponse` (번호 · 이메일 · 닉네임만)
⑥	FastAPI	201 + JSON. 요청이 끝나면 `get_db()` 가 세션을 닫음

결과	상태	예
가입됨	201	`{"user_id": 92, …}`
칸 빠짐 · 모양 틀림	422	`details: {"privacy_agreed": "Field required"}`
비밀번호 규칙 · 동의	422	`details: {"password": ["upper", …]}`
이메일 · 닉네임 사용 중, 탈퇴 3일 안	409	`code: "EMAIL_TAKEN"` 등

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import 이다 (P1-05 로그인 것까지 함께 있음). **`from app.services import auth_service` 를 이번에 추가했다.** 이 함수는 `APIRouter`(→ `router`) · `status` · `Db` · `SignupRequest` · `SignupResponse` · `ErrorResponse` · `auth_service` 를 쓴다.

import	쓰는 곳
`router = APIRouter()`	`@router.post(…)` — 이 파일의 주소들을 묶음. `api/v1/router.py` 가 `/auth` 아래에 붙임
`status.HTTP_201_CREATED`	성공 상태 201
`Db`	DB 세션 (P1-03 `deps.py`)
`SignupRequest` · `SignupResponse`	요청 · 응답 모양
`ErrorResponse`	API 문서(`/docs`)에 409 · 422 응답 모양을 보여 줌
`auth_service`	21 의 `signup()` 을 `auth_service.signup` 으로 부름

함수

```
@router.post(
    "/signup",
    response_model=SignupResponse,
    status_code=status.HTTP_201_CREATED,
    responses={409: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def signup(data: SignupRequest, db: Db) -> SignupResponse:
```

```
    user = auth_service.signup(db, data)
    return SignupResponse(user_id=user.user_id, email=user.email, nickname=user.nickname)
```

위의 `@router.post(…)` 부분은 템플릿에 있었고, 함수 몸통 두 줄을 이번에 채웠다.

문법 풀이

import 문법	뜻
`from app.services import auth_service`	**모듈(파일) 전체**를 이름 하나로 가져옴 → `auth_service.signup` 처럼 씀. 이 파일의 라우터 함수 이름도 `signup` 이라, 서비스 함수를 `from … import signup` 으로 가져오면 이름이 겹친다

식	뜻
`@router.post("/signup", …)`	데코레이터 — 아래 함수를 `POST /signup` 요청에 연결
`response_model=SignupResponse`	응답을 이 모양으로 **걸러서** 보냄. 다른 칸이 있어도 나가지 않음
`status_code=201`	성공하면 200 이 아니라 201(만들어짐)
`responses={409: …, 422: …}`	응답 설명을 API 문서에만 추가 (동작에는 영향 없음)
`data: SignupRequest`	요청 본문 JSON 을 이 모양으로 읽어 넣어 줌 (①)
`db: Db`	DB 세션을 넣어 줌 (②)

4. 흐름

1. FastAPI 가 본문을 검사하고 DB 세션을 연다.
2. 21 `signup()` 으로 가입한다.
3. 번호 · 이메일 · 닉네임만 담아 201 로 응답한다.

5. 왜 이렇게

선택	이유
라우터는 두 줄	검사 · 저장은 서비스(21)에 둔다. 라우터가 얇으면 HTTP 없이도 서비스를 시험할 수 있다
`SignupResponse` 로 바꿔서 응답	`User` 를 그대로 내보내면 `password_hash` 같은 칸이 섞일 위험이 있다. 응답 모양을 정해 두면 그 칸만 나간다 (PRD 01 §5)
201	REST 관례 — 새로 만들었다는 뜻. 화면은 201 로 "가입 성공" 을 안다
오류는 여기서 잡지 않음	21 이 던진 `AppError` 를 01 처리기가 받아 모양을 맞춘다. 라우터마다 `try` 를 쓰지 않아도 된다
가입 후 토큰을 주지 않음	로그인은 P1-05 의 일이다. 가입과 로그인을 나눠 두면 각자 단순하다

6. 시험 (`tests/test_auth_routes.py` 에 4개 추가 · 모두 6개, 실제 앱 + 실제 MySQL · rollback)

시험에서 앱의 `get_db` 를 시험용 세션으로 바꿔 끼워(`app.dependency_overrides`), 진짜 주소로 요청했다.

시험	확인한 것
가입	201, 응답 칸이 정확히 `user_id` · `email` · `nickname` (비밀번호 · 해시 없음), 이메일 소문자, DB 에 있음
같은 이메일로 다시	두 번째는 409 `EMAIL_TAKEN`

시험	확인한 것
비밀번호 규칙	422, `details.password = ["upper", "number", "special"]`
칸 빠짐	422, `details = {"privacy_agreed": "Field required"}` (02 처리기)

전체 시험 149개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.

Backend(FastAPI/Python) 23 — 라우터 `GET /api/v1/auth/check-email`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `check_email`) · `requirements.txt`
작업	P1-04 회원가입 — 이메일 실시간 중복 확인 API · PRD 01 FR-101 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	22 `POST /auth/signup` · 24 `GET /auth/check-nickname`

1. 하는 일

주소의 `?email=` 값을 받아 **형식을 검사**하고, 19 `check_email()` 의 결과를 그대로 돌려준다.

요청	응답
`GET /api/v1/auth/check-email?email=free@example.com`	200 `{"available": true, "reason": null, "available_at": null}`
`…?email=ROUTE.user@example.COM` (이미 가입)	200 `{"available": false, "reason": "TAKEN", …}`
`…?email=gone@example.com` (탈퇴 1일)	200 `{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "2026-10-09T03:00:00Z"}`
`…?email=abc` (형식 틀림)	**422** `{"error": {"code": "VALIDATION_ERROR", "details": {"email": "value is not a valid email address: An email address must have an @-sign."}}}`
`…` (`email` 없음)	422

2. 언제 쓰나

join.html 이메일 칸 옆 **Check** 버튼 → `join.js`(P1-07) 가 이 주소로 묻는다.

응답	이메일 칸 아래 문구 (P1-07)
`available: true`	✓ Available
`TAKEN`	This email is already in use.
`REJOIN_TOO_SOON`	You can sign up again after Oct 9. (`available_at` 을 사용자 시간대로)
422	Enter a valid email address.

형식이 틀리면 422 인 이유 (2026-10-07 결정): "이미 사용 중"(200 · `available: false`)과 "형식이 틀림"(422)을 구분해 다른 문구를 보여 준다. 가입(22)도 같은 `EmailStr` 규칙이라, 여기서 통과한 이메일은 가입에서도 형식으로 막히지 않는다.

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import 이다. **`from pydantic import EmailStr` 를 이번에 추가했다.** 이 함수는 `Annotated` · `Query` · `EmailStr` · `Db` · `AvailabilityResponse` · `auth_service` 를 쓴다.

`requirements.txt` 에 한 줄 추가 (새 패키지)

```
email-validator>=2.2       # pydantic EmailStr — 이메일 형식 검사 (P1-04, 2026-10-07)
```

`EmailStr` 은 pydantic 에 있지만, 실제 검사는 이 패키지가 한다. 설치된 버전은 2.3.0. 다른 컴퓨터는 pull 뒤 의존성 다시 설치(CLAUDE.md)로 자동 설치된다.

함수

```
@router.get("/check-email", response_model=AvailabilityResponse)
def check_email(db: Db, email: Annotated[EmailStr, Query(max_length=255)]) -> AvailabilityResponse:
    return auth_service.check_email(db, email)
```

템플릿은 `Annotated[str, …]` 였고, `str` 을 `EmailStr` 로 바꾸고 몸통 한 줄을 채웠다.

문법 풀이

import 문법	뜻
`from pydantic import EmailStr`	pydantic 의 "이메일 글자" 타입을 가져옴

식	뜻
`@router.get("/check-email", …)`	`GET /api/v1/auth/check-email` 에 연결. `GET` = 보기만 하는 요청
`Annotated[EmailStr, Query(max_length=255)]`	이 인자는 **주소의 `?email=`** 에서 읽고(`Query`), 255자 이하이며 이메일 형식(`EmailStr`)이어야 함. 틀리면 FastAPI 가 라우터 실행 전에 422 (02 처리기)
`auth_service.check_email(db, email)`	19 를 `release=False`(기본값)로 부름 → DB 를 바꾸지 않음
`response_model=AvailabilityResponse`	응답 모양. `available_at` 은 UTC 시각이라 `…Z` 로 나감

`EmailStr` 이 받는 것 · 막는 것 (직접 확인)

입력	결과
`kim@naver.com`	통과
`" kim@naver.com "`	통과 — 앞뒤 공백을 지워 줌
`Kim@Naver.COM`	통과 → `Kim@naver.com` (도메인만 소문자. `@` 앞은 17 이 소문자로)
`abc` · `a b@x.com` · `kim@naver` · `user@localhost`	422
`deleted+85@withdrawn.invalid`	422 — `.invalid` 는 쓸 수 없는 도메인. **18 에서 비켜 둔 이메일로는 누구도 가입할 수 없다**
`한글@naver.com`	통과 — 국제 메일 규격(SMTPUTF8)상 맞는 형식

4. 흐름

1. FastAPI 가 `?email=` 을 읽어 길이 · 형식을 검사한다. 틀리면 422.
2. 19 로 쓸 수 있는지 확인한다 (DB 는 바꾸지 않음).
3. 결과를 200 으로 돌려준다.

5. 왜 이렇게

선택	이유
`GET` · 확인만 (`release=False`)	19 에서 정한 대로. `GET` 은 여러 번 불려도 DB 가 바뀌면 안 된다
"쓸 수 없음" 도 200	요청은 성공했고 답이 "아니요" 인 것이다. 오류(4xx)는 요청 자체가 잘못됐을 때(형식 틀림)
형식 검사를 라우터 입구 (`EmailStr`)에서	틀린 글로 DB 에 묻지 않는다. 오류 모양도 02 처리기가 다른 입력 오류와 똑같이 만든다
`email-validator` 패키지 추가	정규식으로 흉내 내면 `a..b@x.com`, 국제 도메인 같은 경우를 놓친다. 이메일 규격을 따르는 검사를 쓴다 (2026-10-06 사용자 결정 "EmailStr")

6. 시험 (`tests/test_auth_routes.py` 에 10개 추가 · 모두 16개, 실제 앱 + 실제 MySQL · rollback)

시험	확인한 것
없는 이메일	200, `{"available": true, "reason": null, "available_at": null}`
가입된 이메일을 대소문자 섞어 물음	`TAKEN`
탈퇴 1일	`REJOIN_TOO_SOON`, `available_at` 이 `Z`(UTC)로 끝남
형식 틀림 5개	`abc` · `a b@x.com` · `kim@naver` · `deleted+85@withdrawn.invalid` · 빈 글자 → 422 `VALIDATION_ERROR`, `details` 에 `email`
`email` 없음	422
탈퇴 5일 행이 있을 때	`available: true` 지만 그 행의 이메일은 **그대로** (GET 은 확인만)

전체 시험 159개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 24 — 라우터 `GET /api/v1/auth/check-nickname`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `check_nickname`)
작업	P1-04 회원가입 — 닉네임 실시간 중복 확인 API · PRD 01 FR-101 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	23 `GET /auth/check-email` · 25 입력 스키마

1. 하는 일

주소의 `?nickname=` 값을 받아 **길이를 검사**하고, 20 `check_nickname()` 의 결과를 그대로 돌려준다. 23 과 같은 모양이다.

요청	응답
`GET /api/v1/auth/check-nickname?nickname=fresh_nick`	200 `{"available": true, …}`
`…?nickname=ROUTE_USER` (`route_user` 가 이미 있음)	200 `{"available": false, "reason": "TAKEN", …}`
`…?nickname=a` (1자) · 21자 · 빈 글자	**422** `details.nickname`

2. 언제 쓰나

화면	버튼
join.html (P1-07)	닉네임 칸 옆 **Check**
userInfo.html (P1-10)	닉네임 바꾸기 전 확인 — 같은 API 를 씀

응답에 따른 문구는 23 과 같다 (✓ Available / This nickname is already in use. / You can use it again after …).

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    NICKNAME_MIN,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
```

```
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import 이다. **`NICKNAME_MIN` 을 이번에 추가했다.** 이 함수는 `Annotated` · `Query` · `Db` · `NICKNAME_MIN` · `NICKNAME_MAX` · `AvailabilityResponse` · `auth_service` 를 쓴다.

함수

```
@router.get("/check-nickname", response_model=AvailabilityResponse)
def check_nickname(
    db: Db,
    nickname: Annotated[str, Query(min_length=NICKNAME_MIN, max_length=NICKNAME_MAX)],
) -> AvailabilityResponse:
    return auth_service.check_nickname(db, nickname)
```

템플릿은 `Query(max_length=NICKNAME_MAX)` 만 있었다. **`min_length=NICKNAME_MIN` 을 추가**하고 몸통 한 줄을 채웠다.

문법 풀이

import 문법	뜻
`from app.schemas.auth import (…, NICKNAME_MIN, …)`	괄호 안에 한 줄에 하나씩 적은 여러 이름을 가져옴

식	뜻
`Query(min_length=2, max_length=20)`	주소의 `?nickname=` 에서 읽고, 2 ~ 20자여야 함. 틀리면 라우터 실행 전에 422 (02 처리기)
`auth_service.check_nickname(db, nickname)`	20 을 `release=False`(기본값)로 부름 → DB 를 바꾸지 않음

4. 흐름

1. FastAPI 가 `?nickname=` 을 읽어 길이를 검사한다. 틀리면 422.
2. 20 으로 쓸 수 있는지 확인한다 (앞뒤 공백 제거, 대소문자 무시, DB 는 바꾸지 않음).
3. 결과를 200 으로 돌려준다.

5. 왜 이렇게

선택	이유
`min_length` 추가	템플릿대로면 `"a"`(1자)를 물으면 "✓ Available" 이 나오지만, 가입(22)은 2자 미만이라 422 로 막는다. 확인과 가입의 규칙이 같아야 "쓸 수 있다더니 가입이 안 된다" 가 생기지 않는다
나머지는 23과 같음	확인만(`GET`), "쓸 수 없음" 도 200, 형식 · 길이 오류만 422

남은 차이 (25 에서 고침): 지금 길이는 앞뒤 공백을 **지우기 전** 글자로 잰다. `" a "` 는 5자라 통과한 뒤 `"a"` 로 확인된다. 25 에서 "공백을 먼저 지우고 길이를 재는" 닉네임 타입을 만들어, 가입 스키마와 이 라우터가 함께 쓰게 한다.

6. 시험 (`tests/test_auth_routes.py` 에 8개 추가 · 모두 24개, 실제 앱 + 실제 MySQL · rollback)

시험	확인한 것
없는 닉네임	200, `available: true`
가입된 `route_user` 를 3가지로 물음	`route_user` · `ROUTE_USER` · `" route_user "` 모두 `TAKEN`
길이 틀림 3개	1자 · 21자 · 빈 글자 → 422, `details` 에 `nickname`
`nickname` 없음	422

전체 시험 167개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 25 — 입력 스키마 `SignupRequest` · `Nickname`

항목	내용
파일	`studionx/backend/app/schemas/auth.py` · `studionx/backend/app/api/v1/auth.py`
작업	P1-04 회원가입 — 가입 입력 형식 검사 · PRD 01 FR-101 (P1-04 마지막)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	24 `GET /auth/check-nickname` · 26 `create_access_token()` (P1-05)

1. 하는 일

가입 요청(22)과 닉네임 확인(24)의 **입력 모양**을 정한다. 형식이 틀리면 라우터에 닿기 전에 FastAPI 가 422 로 돌려보낸다.

칸	바꾸기 전 (템플릿)	바꾼 뒤
`email`	`str` (아무 글자)	**`EmailStr`** — 이메일 형식 · 앞뒤 공백 제거 · 254자 이하
`nickname`	`str` 2 ~ 20자 (공백 포함 길이)	**`Nickname`** — 앞뒤 공백을 **먼저 지우고** 2 ~ 20자

입력	바꾸기 전	바꾼 뒤
`nickname: " a "`	통과 (5자) → 저장은 `"a"` (1자)	**422** (공백 빼면 1자)
`nickname: " ab "`	통과 → 서비스가 `"ab"` 로	통과 → 스키마가 `"ab"` 로
`email: "kim@naver"`	통과 → DB 에 이상한 이메일	**422**
`email: " kim@naver.com "`	통과 → 서비스가 공백 제거	통과 → 스키마가 공백 제거

2. 언제 쓰나

쓰는 곳	어떻게
22 `POST /auth/signup`	`data: SignupRequest` — 본문 JSON 을 이 모양으로 검사
24 `GET /auth/check-nickname`	`nickname: Annotated[Nickname, Query()]` — 주소의 `?nickname=` 을 같은 타입으로 검사
23 `GET /auth/check-email`	이미 `EmailStr` 사용 (23 에서 바꿈)

확인 · 가입이 같은 타입을 쓰는 이유: 중복 확인에서 "✓ Available" 이라고 했는데 가입에서 형식 오류가 나면 안 된다. 같은 타입이면 규칙이 한 곳에만 있다.

3. 코드

import (`schemas/auth.py` 맨 위)

```
from datetime import datetime
from typing import Annotated
```

```
from pydantic import BaseModel, EmailStr, Field, StringConstraints
```

`Annotated` · `EmailStr` · `StringConstraints` 를 이번에 추가했다.

import	가져오는 것	쓰는 곳
`Annotated` (파이썬 기본)	타입에 조건을 덧붙이는 도구	`Nickname`
`EmailStr`	이메일 형식 타입 (23 에서 설치한 `email-validator` 가 검사)	`email`
`StringConstraints`	글자 조건 묶음 (공백 제거 · 길이)	`Nickname`

함수 (`schemas/auth.py`)

```
NICKNAME_MIN, NICKNAME_MAX = 2, 20

# 닉네임 입력 타입 — 앞뒤 공백을 먼저 지우고 길이를 잼 (가입 · 중복 확인이 함께 씀, 2026-10-07)
Nickname = Annotated[
    str,
    StringConstraints(strip_whitespace=True, min_length=NICKNAME_MIN, max_length=NICKNAME_MAX),
]


class SignupRequest(BaseModel):
    email: EmailStr = Field(max_length=255)  # 형식 검사 · 앞뒤 공백 제거 (email-validator)
    nickname: Nickname
    password: str = Field(min_length=1, max_length=100)  # 정확한 8~20자 검사는 check_password_rules
    privacy_agreed: bool
    privacy_version: str = Field(max_length=20)
```

(`api/v1/auth.py` 의 24 라우터)

```
    nickname: Annotated[Nickname, Query()],     # 바꾸기 전: Annotated[str, Query(min_length=…, max_lengt
h=…)]
```

24 에서 넣었던 `min_length` · `max_length` 는 `Nickname` 안으로 들어가서, 라우터의 `NICKNAME_MIN` · `NICKNAME_MAX` import 는 지웠다.

문법 풀이

import 문법	뜻
`from pydantic import BaseModel, EmailStr, Field, StringConstraints`	pydantic 에서 네 이름을 한 줄로 가져옴

식	뜻
`Nickname = Annotated[str, StringConstraints(…)]`	**타입에 이름을 붙임** — "조건이 붙은 글자" 를 `Nickname` 한 단어로 여러 곳에서 씀
`strip_whitespace=True`	검사 **전에** 앞뒤 공백을 지움. 지운 값이 그대로 `data.nickname` 에 들어감
`min_length` · `max_length`	공백을 지운 **뒤** 길이
`EmailStr = Field(max_length=255)`	이메일 형식 + 255자 이하. 실제로는 이메일 규격 상한 254자에서 먼저 막힘
`Annotated[Nickname, Query()]`	같은 타입을 주소 값(`?nickname=`)에도 씀

4. 흐름 (요청이 들어올 때)

1. FastAPI 가 JSON · 주소 값을 꺼낸다.
2. `email` → 앞뒤 공백 제거 → 형식 검사. `nickname` → 앞뒤 공백 제거 → 2 ~ 20자 검사.
3. 하나라도 틀리면 02 처리기가 422 + `details` (어느 칸인지).
4. 모두 맞으면 정리된 값(공백 지운 이메일 · 닉네임)으로 라우터 → 서비스(21).

5. 왜 이렇게

선택	이유
형식 검사는 스키마(입구)에서	틀린 입력이 서비스 · DB 까지 가지 않는다. 오류 모양도 다른 입력 오류와 같다(02)
비밀번호는 스키마에서 1 ~ 100자만	8 ~ 20자 · 조건 검사는 14 가 한다. 조건별 실패 목록(`["upper", …]`)을 화면에 돌려주려면 서비스에서 해야 한다
이메일 소문자는 스키마가 아니라 17 에서	로그인(P1-05) · 중복 확인도 17 을 거친다. 비교 기준을 한 함수에 모은다
서비스(21)의 `strip()` · `normalize_email()` 은 그대로 둠	스키마를 거치지 않고 서비스를 부르는 곳(시험 · 나중의 관리 기능)이 있어도 같은 기준이 지켜진다
`Nickname` 을 `schemas/auth.py` 에	P1-08 회원 정보 수정(`schemas/user.py`)도 닉네임을 바꾼다. 그때 이 타입을 가져다 쓰면 된다

6. 시험

`tests/test_auth_schemas.py` (새로 만듦 · 16개)

시험	확인한 것
닉네임 통과 4개	`beatmaker` · `" beatmaker "` → `beatmaker` · `" ab "` → `ab` · 20자
닉네임 실패 5개	1자 · `" a "` · 공백만 · 21자 · 공백 빼도 21자 → `nickname` 오류
이메일 공백	`" kim@naver.com "` → `kim@naver.com`
이메일 실패 5개	`abc` · `a b@x.com` · `kim@naver` · `deleted+85@withdrawn.invalid` · 빈 글자 → `email` 오류
이메일 길이	254자는 통과, 300자 넘는 것은 `email` 오류

`tests/test_auth_routes.py` 에 3개 추가 (모두 27개)

시험	확인한 것
`GET /check-nickname?nickname= a`	422 (24 에서 남았던 차이가 고쳐짐)
`POST /signup` 닉네임 `" a "`	422, `details` 에 `nickname`
`POST /signup` 이메일 `kim@naver`	422, `details` 에 `email`

전체 시험 186개 통과, `ruff` 검사 통과.